



**Faculty of Electrical Engineering
Department of Measurement**

Bachelor's Thesis

LionKey (security key) NFC support

Matyáš Martan

Open Informatics - Internet of Things

April 2026

<https://github.com/Matesin/bachelor-thesis>

Supervisor: Ing. Jan Sobotka, PhD.

I. Personal and study details

Student's name: **Martan Matyáš** Personal ID number: **525903**
Faculty / Institute: **Faculty of Electrical Engineering**
Department / Institute: **Department of Measurement**
Study program: **Open Informatics**
Specialisation: **Internet of Things**

II. Bachelor's thesis details

Bachelor's thesis title in English:

LionKey (security key) NFC support

Bachelor's thesis title in Czech:

LionKey (bezpečnostní klíč) rozšíření NFC

Name and workplace of bachelor's thesis supervisor:

Ing. Jan Sobotka, Ph.D. Department of Measurement FEE

Name and workplace of second bachelor's thesis supervisor or consultant:

Date of bachelor's thesis assignment: **09.02.2026** Deadline for bachelor thesis submission: _____

Assignment valid until: **by the end of summer semester 2026/2027**

Head of department's signature

Vice-dean's signature on behalf of the Dean

III. Assignment receipt

The student acknowledges that the bachelor's thesis is an individual work.
The student must produce his thesis without the assistance of others, with the exception of provided consultations.
Within the bachelor's thesis, the author must state the names of consultants and include a list of references.

Date of assignment receipt

Martan Matyáš

Student's signature

I. Personal and study details

Student's name: **Martan Matyáš**

Personal ID number: **525903**

Faculty / Institute: **Faculty of Electrical Engineering**

Department / Institute: **Department of Measurement**

Study program: **Open Informatics**

Specialisation: **Internet of Things**

II. Bachelor's thesis details

Bachelor's thesis title in English:

LionKey (security key) NFC support

Bachelor's thesis title in Czech:

LionKey (bezpečnostní klíč) rozšíření NFC

Guidelines:

For the LionKey project: An open-source FIDO2 USB Security Key, implement basic support for NFC technology.

The work consists of the following parts:

- 1) Familiarization with the existing codebase and the technologies needed to implement NFC.
- 2) Implement basic NFC functionality on the STEVAL-ST25R3916B hardware.
- 3) Demonstrate basic functionality - e.g., authentication on <https://webauthn.io>

Bibliography / sources:

<https://github.com/pokusew/lionkey>

https://dspace.cvut.cz/bitstream/handle/10467/123886/F3-DP-2025-Endler-Martin-FIDO2_USB_Security_Key.pdf?sequence=1&isAllowed=

Add files generated by KOS

Acknowledgement

Abstrakt / Abstract

Tato bakalářská práce se zabývá implementací podpory technologie NFC pro hardwarový bezpečnostní klíč LionKey, který slouží jako externí FIDO2 autentizátor. Cílem práce je rozšířit existující zařízení o komunikační rozhraní umožňující jeho použití také se zařízeními vybavenými NFC čtečkou, zejména pak s mobilními telefony.

Práce popisuje teoretické principy technologie NFC, související komunikační standardy a jejich vztah k použití bezpečnostních klíčů v autentizačních mechanismech. Dále se věnuje návrhu a realizaci NFC rozhraní pro platformu LionKey, integraci do existující architektury klíče a ověření funkčnosti výsledného řešení.

Výsledkem práce je funkční implementace NFC komunikace pro klíč LionKey s využitím platformy STEVAL-D25R16B. Výsledné řešení je veřejně dostupné na GitHubu a rozšiřuje možnosti použití zařízení na stránkách podporujících WebAuthn.

Klíčová slova: NFC, WebAuthn, CTAP2, bezpečnostní klíč

Překlad titulu: LionKey (bezpečnostní klíč) rozšíření NFC

This bachelor's thesis focuses on implementing NFC support for the LionKey hardware security key, which serves as an external FIDO2 external authenticator. The aim of this thesis is to extend the capabilities of the existing device by implementing an NFC communication interface on top of the existing solution, allowing the key to communicate with devices equipped with an NFC reader, especially mobile phones.

The thesis describes the theoretical principles of NFC technology, the related communication standards, and their relevance to the use of security keys in authentication mechanisms. It also focuses on the design and implementation of an NFC interface for the LionKey platform, its integration into the existing device architecture, and the verification of the resulting solution.

The result of this thesis is a functional implementation of NFC communication for the LionKey security key using the STEVAL-D25R16B platform. The resulting implementation is publicly available on GitHub. It extends the device's capabilities by allowing its use with services supporting WebAuthn.

Keywords: NFC, WebAuthn, CTAP2, security key

Contents /

1 Introduction	1	
1.1 Thesis Objective	1	
1.2 Thesis Structure	1	
2 An Alternative Authentication Method	2	
2.1 Passkeys	2	
2.1.1 Hardware security keys	3	
2.2 The FIDO2 Framework	3	
2.2.1 Post-Quantum Security of the CTAP Protocol	4	
3 NFC	5	
3.1 Introduction to NFC	5	
3.2 Device Types	5	
3.3 Device Roles	6	
3.4 Operating Modes	7	
3.5 NFC RF Technologies	8	
3.5.1 NFC-A	8	
3.5.2 NFC-B	8	
3.5.3 NFC-F	8	
3.6 NFC-A State Machine	9	
3.6.1 NFC-A Initialization and Anticollision	10	
3.7 Communication Protocols	12	
3.7.1 ISO-DEP	12	
3.7.2 NFC-DEP	16	
4 Implementation	17	
4.1 NFC technologies used	17	
4.2 Hardware	17	
4.2.1 Board Specifications	18	
4.2.2 Note: Energy Harvesting	18	
4.3 Middleware	18	
4.4 Communication with the NFC Frontend	19	
4.4.1 SPI Interface	19	
4.4.2 Interrupt Handling	19	
4.5 Software Implementation	20	
4.5.1 Transaction Handling	20	
5 Results and Evaluation	22	
5.1 Methodology	22	
5.2 Response Testing	23	
5.3 Communication Capabilities Testing	23	
5.3.1 CTAP Handling	23	
5.3.2 NDEF Handling	25	
5.3.3 Use of CTAP on Real WebAuthn-enabled Websites	25	
6 Conclusion	28	
References	29	
A Glossary	31	

Tables / Figures

3.1	ATQA message structure	11	3.1	NFC Protocol Stack.....	7
3.2	ANTICOLLISION command structure.....	12	3.2	NFC Initialization Flow	10
3.3	SAK bit interpretation.....	12	3.3	FIDO Select Log	15
3.4	FSDI to FSC conversion	13	4.1	STEVAL-D25R3916B board...	18
3.5	RATS message structure.....	13	4.2	RFAL Application Overview ..	19
3.6	ATS message structure	13	4.3	STEVAL-D25R3916B con- nection scheme	20
3.7	Short C-APDU byte layout	14	4.4	Implemented State Machine ...	21
3.8	Extended C-APDU byte lay- out	15	5.1	NFC Card Details.....	23
3.9	R-APDU structure.....	15	5.2	CTAP_GETINFO Log	24
			5.3	NDEF Handling Log	26
			5.4	Webauthn Authentication Flow (1)	26
			5.5	Webauthn Authentication Flow (2)	27

Chapter 1

Introduction

Passkeys have become a **popular means of authentication** on the Internet. Along with more conventional means of passkeys, users are also adopting **hardware security keys** as a **safer way to store their credentials**. Hardware security keys may provide an additional layer of security compared to platform authenticators. They offer stronger isolation of private keys and ensure their non-exportability, thereby reducing the risk of credential compromise even if the host device is not trusted.

One such project is the **LionKey**, an open-source hardware security key developed at **CTU FEE** by **Ing. Martin Endler**. The device is compliant with FIDO2 standards; however, it currently **supports only CTAP over USB** (CTAPHID). This means that the key cannot be used with devices that do not support USB-based communication, notably mobile phones.

1.1 Thesis Objective

The objective of this thesis is to **extend** the **communication capabilities** of the LionKey hardware security key by **implementing an NFC-based communication** interface on top of the existing CTAPHID solution, in order to further improve user experience when using the authenticator.

The NFC interface is realized using the STEVAL-D25R3916B board, which is interfaced with the ST Nucleo-H533RE microcontroller in accordance with the original specifications of the LionKey project.

1.2 Thesis Structure

The thesis is organized into six main parts:

The **Introduction** outlines the context of the work and formulates the objectives of the thesis.

Chapter 2 presents the theoretical background of passkeys and motivates their use in modern authentication systems.

Chapter 3 introduces the fundamental principles of NFC technology, focusing on aspects relevant to the proposed solution.

Chapter 4 details the design, hardware used, and implementation of the proposed solution.

Chapter 5 describes the results achieved by this implementation and its applicability to practice.

Chapter 6 summarizes the results achieved and discusses the general outcome of the thesis.

Chapter 2

An Alternative Authentication Method

This chapter defines the motivation for the use of an alternative authentication method.

The Internet has become an integral part of daily life for most of the world's population. In December 2025, approximately **6.047 billion people** (73.2% of the global population) were **Internet users**, representing a 14% increase compared to 2020 [1]. This growth, along with an increasing reliance on online services, has been accompanied by an increase in cyberattacks and personal data breaches.

This development highlights a fundamental weakness of password-based authentication. According to the Verizon Data Breach Investigations Report, approximately **88% of breaches** in 2025 involved **stolen credentials** [2]. Passwords are therefore inherently vulnerable to theft, reuse, and brute-force attacks.

In response, organizations have **increasingly adopted multi-factor authentication (MFA)**, which supplements passwords with additional verification factors such as one-time codes, authentication applications, or trusted devices. The adoption of MFA has grown significantly; for example, **83% of organizations required MFA** for all access to IT in 2024 [3]. Despite variations across surveys, it is clearly observable that **MFA has become a popular safety measure**.

2.1 Passkeys

Despite the growing usage of MFA, Ing. Martin Endler argues in his diploma thesis that many **existing authentication methods** “**do not provide sufficient security** and hinder the user experience” [4]. Passkeys – standardized by the FIDO Alliance – have emerged as a response to these limitations.

The FIDO Alliance **defines a passkey** as “...[an] authentication **credential** based on FIDO standards that allows a user to **log into apps and websites** with the same process they use to unlock their device (biometrics, PIN or pattern). Passkeys rely on **asymmetric cryptography**, where the private key is securely stored on the user's authenticator, while the corresponding public key is registered with the service ” ¹.

According to data published by the FIDO Alliance, passkey support has already been **adopted** by approximately a **fifth** of the world's **top 100 websites** and services [5]. Furthermore, major technology providers, including Apple, Google, and Microsoft, have integrated passkey support into their platforms, allowing further support for this authentication method.

¹ <https://fidoalliance.org/passkeys/>

■ 2.1.1 Hardware security keys

Although passkeys are often implemented as platform authenticators integrated into user devices, the **FIDO2** framework also supports **external authenticators** in the form of **hardware security keys**. These devices **store cryptographic credentials** on dedicated hardware and **perform authentication independently** of the host system and only **upon user interaction**, providing an additional layer of security.

Currently, the hardware security key market is dominated by several major vendors, most notably **Yubico** with its Security Key Series. Other notable solutions include the Google Titan Security Key developed by Google and the SecurID authenticators made by RSA Security. In addition, several smaller or less widely cited manufacturers provide compatible FIDO2 security keys, including FEITIAN Technologies, Swissbit, and TrustKey.

Quantitative estimates of market share are difficult to obtain, as publicly available data are limited and often based on proprietary methodologies. **Estimates suggest**, as of April 2026 [6] that **Yubico accounts for approximately 32% and RSA Security for 24%** of the two-factor authentication **solutions**. However, these figures reflect only the customer base observed by the source platform and should be interpreted as indicative rather than representative of the global market share of hardware security keys.

A distinguishing **characteristic** of most commercially available hardware security keys is their **closed-source firmware**. Open-source alternatives do exist but remain rare: the first publicly known example was SoloKeys (developed 2018–2024, now discontinued), followed by OpenSK by Google and Nitrokey. To the best of our knowledge, no open-source C-based implementation of CTAP2 over NFC for a hardware security key was publicly available at the time of writing.

■ 2.2 The FIDO2 Framework

The **FIDO2** framework, developed by the FIDO Alliance in collaboration with the W3C, is the current **industry standard** for **strong, phishing-resistant and passwordless authentication**. It is an open framework that uses public-key cryptography.

FIDO2 is composed of two complementary core specifications:

- **WebAuthn (Web Authentication):** A W3C standard API that enables web applications to create and use public-key credentials. It defines how a browser or platform (the “Client”) communicates with a web service (the “Relying Party”) to register and authenticate users.
- **CTAP (Client-to-Authenticator Protocol):** A FIDO Alliance specification that defines how a Client communicates with an authenticator (such as a hardware security key or a smartphone) to fulfill WebAuthn requests.

The CTAP standard defines two primary versions:

- **CTAP1 (formerly U2F):** Supports legacy FIDO U2F credentials.
- **CTAP2:** The current standard for FIDO2, which improves user verification methods, most notably by implementing a PIN/User Verification (UV) authentication model.

A **detailed breakdown** of this framework, including a detailed description of the CTAP protocol, can be **found** in the **work of Endler** [4], along with a more comprehensive insight into the problematic of hardware security keys, including the cryptographic mechanisms that are commonly used.

■ 2.2.1 Post-Quantum Security of the CTAP Protocol

To further extend Endler’s overview, this section briefly examines the **post-quantum (PQ) security** considerations of the CTAP protocol. As quantum computing technology advances, it poses a significant **long-term threat** to currently used public-key cryptographic schemes [7]. Although no quantum computer is currently capable of breaking them, the issue is important to acknowledge.

The version of CTAP implemented in this project (CTAP 2.2) **does not provide native PQ security**, as it relies on conventional public-key cryptographic primitives that could be compromised by sufficiently capable quantum adversaries. Nevertheless, the current protocol architecture is considered to be **PQ ready**. In particular, its PIN/UV authentication model requires active user verification and physical presence, while the protocol itself can be adapted to stronger cryptographic primitives without a fundamental redesign. Therefore, a full quantum-resistant operation **could be achieved** by replacing the current asymmetric mechanisms with post-quantum key encapsulation and signature mechanisms, provided that such changes are introduced consistently across both CTAP and WebAuthn within the FIDO2 framework [8].

The topic of making the CTAP protocol quantum-safe is **discussed** in more detail in the **work of Kniep** [9].

Chapter 3

NFC

This chapter introduces the principles of **Near Field Communication (NFC)** relevant to the implementation presented in this thesis. Since NFC covers a broad set of radio technologies, operating modes, and higher-layer protocols, the discussion is **limited** primarily to **NFC-A**, **Card Emulation**, **ISO-DEP**, and **APDU-based communication**.

Other technologies within the NFC ecosystem are discussed only briefly to provide context and completeness.

3.1 Introduction to NFC

NFC is a short-range wireless communication technology derived from radio-frequency identification (RFID) systems. It enables **bidirectional data exchange** between devices in immediate or close proximity, typically within a range of up to 10 cm, with reliable operation occurring at distances of approximately 4 cm [10]. It is standardized primarily through ISO/IEC specifications and NFC Forum¹ specifications. The key specifications include ISO/IEC 14443, which defines proximity card communication (used in NFC-A and NFC-B), and ISO/IEC 18092, which specifies peer-to-peer communication. In addition, the NFC Forum defines higher-level protocols and data formats to ensure interoperability across devices.

NFC operates at a **carrier frequency (fc) of 13.56 MHz** and is based on **inductive coupling** between **loop antennas**. In this mechanism, defined in [11], an active device generates an electromagnetic field, while a passive device modulates this field to transmit data.

NFC supports three primary operating modes: reader/writer mode, card emulation (CE) mode, and peer-to-peer mode. From the perspective of this thesis, the CE mode is of particular importance, as it allows the device to behave as a contactless smart card and participate in ISO-DEP communication with an external reader.

3.2 Device Types

NFC devices fall into three categories: **active** devices (proximity coupling devices or PCDs), **passive** devices (proximity integrated circuit cards or PICCs), and **hybrid** devices (proximity extended devices or PXDs).

Active devices, or proximity coupling devices (**PCDs**), are equipped with their own power source and generate an electromagnetic field, enabling them to initiate and control NFC communication.

¹ <https://nfc-forum.org>

In contrast, **passive devices PICCs**, commonly implemented as **dynamic NFC tags**, do **not require an internal power supply**. Instead, they rely on the energy harvested from the electromagnetic field generated by an active device (**Energy Harvesting**). **Communication** from the passive side is achieved through **load modulation**, i.e., by varying the impedance of the antenna circuit, which in turn modulates the carrier field. The characteristics of PICCs are defined in [12].

Hybrid devices, or proximity extended devices (**PXDs**), can operate in either PCD or PICC mode. Such devices may switch between modes automatically depending on the communication context, or allow **manual selection** by the user.

3.3 Device Roles

In NFC communication, two roles are distinguished according to which device initiates the interaction: the **initiator** and the **target**. These roles are defined at the protocol level and are independent of the specific hardware capabilities of the devices.

■ Initiator

The device that actively starts the communication and generates the initial RF field. It is responsible for polling and controlling the communication sequence, as is typically the case for an NFC reader.

■ Target

The device that responds to the initiator. Depending on the communication mode, it can passively modulate the initiator's field or actively generate its own signal. Typical examples include contactless smart cards, NFC tags, or peer devices.

It is important to distinguish these roles from the physical ability to generate an electromagnetic field. In passive communication mode, the initiator generates the RF field. In active communication mode, both devices are capable of generating their own RF field during their respective transmission phases.

NFC defines two communication modes based on how the electromagnetic field is generated and how the devices are powered: **active mode** and **passive mode**.

■ Active Mode

In active mode, both the initiator and the target are equipped with their own power sources and are capable of generating an RF field. Communication is performed through alternating transmission, where each device generates its own carrier field during its transmission phase. This mode is used primarily in peer-to-peer communication scenarios.

■ Passive Mode

In passive mode, only the initiator generates the RF field. The target does not have its own power source and utilizes energy harvesting. Data transmission from the target is achieved through load modulation of the existing carrier field. This mode is characteristic of typical NFC-A and NFC-B interactions, such as communication with contactless smart cards and NFC tags.

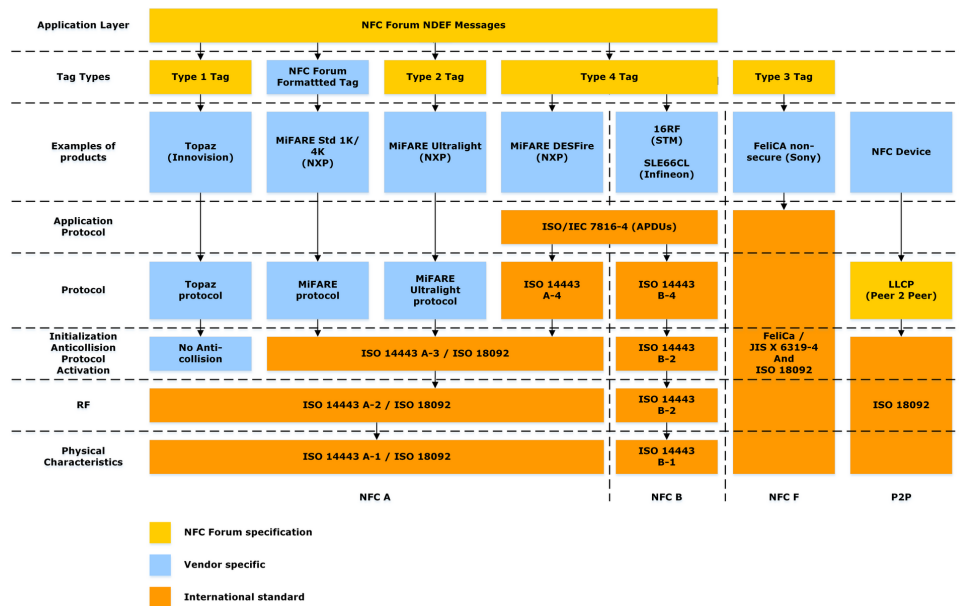


Figure 3.1. NFC Protocol Stack [13]

3.4 Operating Modes

The NFC Forum defines **three** primary **operating modes** that determine the role and behavior of a device within a communication session.

Reader/Writer Mode

In the reader/writer mode, the NFC device operates as an initiator and interacts with passive NFC tags. The device generates an electromagnetic field and communicates with tags compliant with standards such as ISO/IEC 14443 (Type 2 and Type 4 Tags) or ISO/IEC 15693 (Type 5 Tags). This mode is commonly used for applications such as reading NDEF records, accessing contactless cards, or configuring embedded devices.

Card Emulation Mode

In CE mode, the NFC device behaves as a contactless smart card and acts as a target. It responds to commands issued by an external reader, typically using ISO/IEC 14443 protocols. This mode is widely used in applications such as contactless payments, access control, and mobile wallets. This mode has two variants of implementation. It can either be implemented on a general-purpose MCU or using a dedicated secure element (SE) hardware, depending on the use case. Commercial NFC-capable security keys commonly rely on secure-element-based designs, as indicated, for example, by the Yubikey tech manual [14] or by the Google Titan documentation [15].

Peer-to-Peer Mode

The peer-to-peer mode enables half-duplex communication between two NFC-enabled devices, where both devices can alternately assume the roles of initiator and target. Communication in this mode is defined by ISO/IEC 18092 and the NFC Forum Logical Link Control Protocol (LLCP). It is primarily used for data exchange scenarios such as device pairing or file transfer, although its practical use has declined in favor of higher-throughput wireless technologies.

These operating modes build upon the previously described concepts of initiator and target roles, as well as active and passive communication modes. Their combination defines the functional behavior of NFC devices in real-world applications.

3.5 NFC RF Technologies

3.5.1 NFC-A

NFC-A is based on ISO/IEC 14443 Type A. In typical NFC-A communication, initialization and anticollision are performed at 106 kbit/s ($f_c/128$). In theory, the specification allows speeds up to 27.12 Mbit/s ($2f_c$) after activation. However,

Encoding and Modulation: NFC-A employs **Miller (delay) encoding**, a line code where logical bits are represented by time-dependent transitions in signal amplitude. This encoding improves robustness against noise and enables reliable clock synchronization between reader and card. The encoded signal then modulates the RF carrier using different schemes depending on transmission speed:

- **106 kbit/s to $f_c/16$:** Amplitude Shift Keying (ASK) modulation, where the carrier amplitude shifts between minimum and maximum to represent encoded bits.
- **$f_c/8$ to $f_c/2$:** Same modulation as NFC-B (see below).
- **$3f_c/4$ to $2f_c$:** Phase Shift Keying (PSK) modulation, where the carrier phase shifts to encode information, reducing sensitivity to amplitude noise at higher speeds.

3.5.2 NFC-B

NFC-B is based on ISO/IEC 14443 Type B. Initialization and anticollision operate at 106 kbit/s, with a maximum data exchange speed of 6.78 Mbit/s ($f_c/2$), lower than NFC-A's maximum.

Encoding and Modulation: NFC-B uses **NRZ-L (Non-Return-to-Zero Level)** encoding, where high and low amplitude levels directly represent logical 1 and 0 without returning to zero between bits. Transmission is **asymmetric**:

- **Reader-to-card:** 10% Amplitude Shift Keying (ASK) modulation, where the carrier amplitude drops to approximately 90% to represent a logical 0, and remains at full amplitude for logical 1. The low modulation index keeps the carrier stable, reducing RF noise pickup.
- **Card-to-reader:** Binary Phase Shift Keying (BPSK) modulation of a subcarrier, which allows the card to respond without being overwhelmed by the reader's transmitted signal. The initial phase of the subcarrier represents logical 1 and a 180 degree shift represents a logical 0.

The asymmetric modulation scheme and lower modulation index result in **improved robustness against electromagnetic interference**—a critical advantage for applications requiring high reliability.

3.5.3 NFC-F

NFC-F is based on the JIS X 6319-4 specification and was originally developed by Sony under the commercial name FeliCa. Unlike NFC-A and NFC-B, NFC-F is not strictly derived from the ISO/IEC 14443 family, but instead follows a different protocol structure defined in Japanese industrial standards.

A key distinguishing feature of NFC-F is its higher data rates, typically 212 kbit/s or 424 kbit/s, which makes it suitable for latency-sensitive applications. The technology is used primarily in Japan, particularly in transportation systems, contactless payment infrastructures, and access control solutions.

However, due to its regional focus and protocol differences, NFC-F is not universally supported by all NFC readers. Consequently, its applicability in general-purpose NFC implementations is limited and it is mentioned in this overview purely for completeness.

3.6 NFC-A State Machine

NFC-A anticollision and activation sequence has a defined state machine behavior, respecting [16]. This state machine must be respected for the correct initialization of the communication and protocol flow.

A PCD can use the following commands during the anticollision sequence an NFC-A PICC:

- **REQA** (REQuest, Type A)
Broadcast command used to poll for any NFC-A cards in the field. Only cards in IDLE state respond.
- **WUPA** (Wake-UP, Type A)
Similar to REQA, but also wakes cards in HALT state. Cards in both IDLE and HALT states respond.
- **ANTICOLLISION**
Used to resolve collisions when multiple cards respond simultaneously, by iteratively isolating individual UIDs.
- **SELECT**
Selects a specific card by its full UID, moving it into the ACTIVE (or ACTIVE*) state.
- **HLTA**
Instructs the active card to enter the HALT state, suspending it until a WUPA command is received.

The NFC-A state machine has the following States:

- **POWER-OFF**
The PICC is not in an RF field exceeding H_{min} . Communication is not possible. The transition to IDLE occurs when the RF field strength exceeds H_{min} .
- **IDLE**
The PICC is powered by the RF field but has not yet been addressed. It responds only to REQA or WUPA commands. A valid REQA or WUPA transitions the card to READY.
- **READY**
The PICC has responded to REQA or WUPA. The anticollision loop begins here: the PCD issues ANTICOLLISION commands to isolate the card's UID, then a SELECT command to uniquely address it. A successful SELECT transitions the card to ACTIVE. A failed SELECT or collision returns the card to IDLE.
- **READY***
Functionally identical to READY, but entered exclusively from HALT via WUPA. The distinction ensures that the PCD can track whether a card was freshly powered or previously halted. A successful SELECT transitions the card to ACTIVE*.
- **ACTIVE**
The PICC has been selected by its full UID and is ready to communicate. If the card supports ISO-DEP 3.7.1, it awaits a RATS command to enter the PROTOCOL

state. Otherwise, it continues with its proprietary protocol. An HLTA command transitions the card to HALT.

■ ACTIVE*

Functionally identical to ACTIVE, but entered from READY*. The distinction allows the PCD to identify cards that were reactivated from HALT, which may be relevant for session management. A HLTA command returns the card to HALT.

■ HALT

The PICC is suspended and ignores all commands except WUPA. This allows the PCD to selectively wake specific cards in a multi-card scenario. A valid WUPA transitions the card to READY*.

■ PROTOCOL

The PICC operates under the ISO-DEP protocol, initiated by a successful RATS/ATS exchange. All subsequent communication follows the negotiated parameters (FWT, FSC, etc.). The card remains in this state until the RF field is removed or a DESELECT command is issued, returning it to HALT or IDLE, respectively.

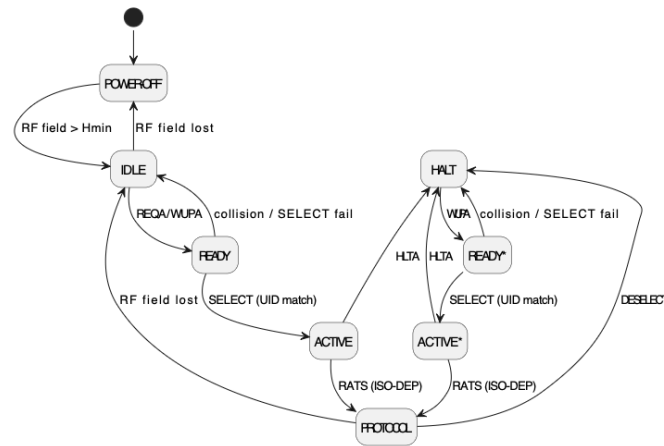


Figure 3.2. PlantUML diagram of the NFC initialization flow.

■ 3.6.1 NFC-A Initialization and Anticollision

NFC-A uses a **bit-frame anticollision** method based on binary tree traversal. Since multiple PICCs may respond simultaneously to a REQA or WUPA command, their signals can overlap and corrupt each other, this is called a **collision**. The anticollision loop resolves collisions iteratively by narrowing down the UID of each card in the field. This overview focuses solely on the NFC-A procedures, although the standard also specifies behavior for hybrid tags supporting both the NFC-A and NFC-B types.

The commands defined in this subsection are later used in Chapter 5 to ensure correct operation of the NFC frontend.

3.6.1.1 Polling

Before sending a REQA command, the PCD must present an unmodulated RF field for a minimum of **5.1 ms**. This ensures passive PICCs, which rely on energy harvesting, are sufficiently powered before they are expected to respond. The PCD then broadcasts REQA to poll all Type A PICCs in IDLE state, or WUPA to additionally wake PICCs in HALT state.

3.6.1.2 Answer To reQuest (ATQA)

Upon receiving a valid REQA or WUPA, all eligible PICCs respond simultaneously with an **ATQA (Answer To reQuest, Type A)**, a 2-byte frame encoding:

- **Bit frame anticollision indicator** - which bit position is used as the anticollision entry point in the binary tree traversal
- **UID size indicator** - whether the UID is 4, 7, or 10 bytes, determining how many cascade levels the anticollision loop must traverse

Byte 1	Byte 2
Bit frame anticollision indicator	UID size indicator

Table 3.1. ATQA structure.

If multiple PICCs respond in the same slot, their ATQA frames collide at the bit level. The PCD detects this and proceeds with the anticollision loop to isolate individual cards.

3.6.1.3 UID Structure and Cascade Levels

NFC-A PICCs have UIDs of three possible lengths, each requiring a different number of cascade levels to fully resolve:

- **Single (4 bytes)** - resolved in one cascade level (CL1)
- **Double (7 bytes)** - resolved across two cascade levels (CL1, CL2)
- **Triple (10 bytes)** - resolved across three cascade levels (CL1, CL2, CL3)

Each cascade level transmits 4 bytes of UID data at a time. When more levels remain, the first byte of the current level is set to the **Cascade Tag (CT, value 0x88)**, signaling to the PCD that the UID continues in a subsequent cascade level.

3.6.1.4 Anticollision Loop

The following sequence repeats for each required cascade level:

- The PCD sends an **ANTICOLLISION** command specifying the current cascade level (CL1 (0x93), CL2 (0x95), or CL3 (0x97)) and an **NVB (Number of Valid Bits)** field indicating how many leading bits of the UID are already known. Initially NVB = 0, meaning no bits are known yet.
- All PICCs in READY (or READY*) state whose UID matches the known prefix respond with their remaining UID bits.
- If exactly one PICC responds, no collision occurred and the PCD has resolved the full UID fragment for this level.
- If multiple PICCs respond simultaneously, a collision is detected at the first bit position where responses differ. The PCD appends either a **0** or **1** at the collision point, increments NVB, and reissues the ANTICOLLISION command - effectively traversing one branch of a binary tree. This repeats until a single PICC is isolated.
- Once the full 4-byte UID fragment for the current level is known, the PCD issues a **SELECT** command containing the complete fragment and a **BCC (Block Check Character)**, which is the XOR of the four UID bytes, used for integrity verification.
- The addressed PICC responds with a **SAK (Select Acknowledge)**, confirming selection. The SAK also indicates whether further cascade levels are required (bit 2 of SAK set) or whether the full UID has been resolved (bit 2 cleared).

SEL	NVB				UID CL _n (partial)	BCC	E
CL(1-3) _y	0	byte count	0	bit count	0–4 bytes (NVB dependent)		

Table 3.2. ANTICOLLISION command structure.

Bit 2	Bit 5	Meaning
1	–	UID incomplete, further cascade levels required
0	0	UID complete, proprietary protocol
0	1	UID complete, ISO-DEP supported

Table 3.3. SAK bit 2 and bit 5 interpretation.

Once all cascade levels are resolved and the final SAK confirms a complete UID, the PICC transitions to **ACTIVE** state. If the SAK additionally indicates ISO-DEP support (bit 5 set), the PCD may proceed with a RATS command to initiate the ISO-DEP protocol. Otherwise, the PICC continues with its own proprietary protocol.

3.7 Communication Protocols

NFC communication is built on a layered protocol stack that defines how data is exchanged between devices after the initial activation phase. At the transport and higher communication layers, two primary protocols are used: ISO-DEP and NFC-DEP. In addition, several auxiliary protocol layers defined by the NFC Forum provide standardized data exchange formats.

3.7.1 ISO-DEP

The **ISO-DEP** (ISO Data Exchange Protocol), specified in [17], defines a transport layer for reliable data exchange between a PCD and a PICC. It extends the lower NFC-A/B protocols by introducing a block-oriented, half-duplex communication mechanism that enables structured and robust transmission of application data. This is the most significant aspect of the NFC technology for the subject of this thesis.

3.7.1.1 Activation and Parameter Negotiation

Before data exchange can begin, ISO-DEP requires an activation phase in which the initiator selects the target and negotiates communication parameters. This is performed using the RATS (Request for Answer To Select) and ATS (Answer To Select) mechanism.

First, the reader sends a RATS (Request for Answer to Select) command. RATS is a 2-byte command consisting of a fixed command byte (E0) and a parameter proposal byte (PP).

RATS consists of a fixed command byte 0xE0 followed by a parameter byte. The upper nibble of the parameter byte encodes the Frame Size for Device Integer (FSDI), while the lower nibble encodes the Card Identifier (CID), which logically addresses a specific

FSDI	0	1	2	3	4	5	6
FSD (bytes)	16	24	32	40	48	64	96
FSDI	7	8	9	A	B	C	D–F
FSD (bytes)	128	256	512	1024	2048	4096	RFU

Table 3.4. FSDI to FSD conversion table

E0	Parameter Proposal		CRC1	CRC2
	FSDI	CID		

Table 3.5. RATS structure.

card when multiple cards are in the field. This value remains assigned to the PICC until it leaves the field. CID=15 is reserved.

The ATS response consists of a total length value (TL), format byte (T0), optional interface bytes (TA, TB, TC) and optional Historical Bytes (up to 15 bytes). These fields define key communication parameters, including timing constraints and frame sizes.

TL	T0	TA(1)	TB(1)	TC(1)	T1	T_k	CRC1	CRC2
		interface				historical bytes		

Table 3.6. ATS structure.

The interface bytes can contain these values:

■ **Frame Waiting Time (FWT)**

Specifies the maximum time the initiator waits for a response from the target. It is derived from the Frame Waiting Time Integer (FWI) in TB(1) and directly influences the allowable processing time of the card. Violating this constraint typically results in termination of the NFC session.

■ **Waiting Time Extension (WTX)**

Allows the target to request additional processing time beyond the configured FWT via a special block. This mechanism is essential for operations involving computationally intensive tasks, such as cryptographic processing.

■ **Frame Size (FSC/FSD)**

Defined by the FSCI field in T0, it specifies the maximum payload size of a single frame for the PICC (FSC) and the PCD (FSD), determining whether chaining is required.

■ **Frame Guard Time (FGT)**

Derived from the Guard Time Integer (GTI) in TB, FGT specifies the **minimum delay the target must observe between receiving a frame and sending its response**. This prevents receiver saturation and ensures reliable synchronization.

■ **Device Identifier (CID)**

When there are multiple cards present in the field, the initiator can use CID (0–14) to address a specific card. If not used, the CID is typically set to 0 or completely omitted.

■ Historical Bytes (HB)

Optional data included in the ATS, they are to be interpreted according to the card manufacturer's specifications, they may provide additional information about the capabilities of the card if present.

Correct configuration and handling of these parameters is essential to ensure compatibility with readers and mobile devices. Strict adherence to timing constraints, such as the frame waiting time, is required to prevent premature session termination.

3.7.1.2 Data Exchange

Communication in **ISO-DEP** is organized into protocol data units (blocks), which are exchanged between the initiator and the target. Three types of blocks are defined: **I-Blocks** to carry application data, **R-Blocks** for acknowledgments and retransmission requests, and **S-Blocks** for control operations such as waiting time extension or protocol deactivation. This structure ensures reliable communication even in the presence of transmission errors.

3.7.1.3 APDU

ISO-DEP serves as the transport layer for **Application Protocol Data Units (APDUs)**, which implement a command-response communication model. APDUs are a standardized data format used in smart card systems for structured information exchange defined by ISO/IEC 7816-4.

In this model, the initiator sends a **command APDU (C-APDU)**, and the target responds with a **response APDU (R-APDU)**. This abstraction allows higher-level protocols, including smart card applications and CTAP over NFC, to operate independently of the underlying transmission mechanism while relying on ISO-DEP for reliable delivery.

A command APDU (C-APDU) consists of a header (CLA, INS, P1, P2) and an optional payload defined by the fields Lc (length of command data) and Le (expected length of response data). The response APDU (R-APDU) may contain response data and is always terminated by a two-byte status word (SW1, SW2), indicating the result of the operation.

APDUs can be encoded in short or extended format. In the short format, the length fields are limited to one byte, allowing payloads of up to 255 bytes. The extended format uses two-byte length fields, enabling payloads of up to 65,535 bytes.

In the short format, the C-APDU consists of the mentioned 4-byte header followed by the optional fields Lc, Data (0–255 bytes) and Le, where both Lc and Le are encoded using a single byte.

CLA	INS	P1	P2	Lc	Data	Le
1 byte	1 byte	1 byte	1 byte	0/1	0–255	0/1

Table 3.7. Byte-level structure of short C-APDUs.

In the extended format, the presence of the leading 0x00 byte indicates extended length encoding, followed by two-byte length fields for Lc and Le. The exact position of Le depends on the APDU case.

CLA	INS	P1	P2	00	Lc(Hi)	Lc(Lo)	Data/Le
1B	1B	1B	1B	ext. marker	length		variable

Table 3.8. Byte-level structure of extended C-APDUs.

Data	SW1	SW2
(optional) response data	status	word

Table 3.9. General structure of a R-APDU.

Response APDUs consist of an optional data field followed by a mandatory two-byte status word (SW1, SW2).

The practical structure of APDU communication can be illustrated on the **following example**:

```

CE: start waiting for command
CE: negotiated max frame size: 252
CE: start RX successful
Received APDU (13 bytes): 00A4040008A0000006472F0001
NFC: APDU parsed successfully
NFC: FIDO AID selected
Sent APDU (10 bytes): 4649444F5F325F309000

```

Figure 3.3. Device log of processing an APDU and selecting the FIDO applet.

C-APDU:	00	A4	04	00	08	A0	00	00	06	47	2F	00	01
						\	_____	Data	_____	/			
	CLA	INS	P1	P2	Lc								
R-APDU:	46	49	44	4F	5F	32	5F	30	90	00			
	\	_____	Data	_____	/	\	SW1	SW2	/				

In this example, the C-APDU consists of the following fields: CLA = 0x00 (interindustry class byte - a standard entry point for commands), INS = 0xA4 (SELECT instruction), P1 = 0x04 (selection by name) and P2 = 0x00. The field Lc = 0x08 specifies the length of the data field, which contains the 8-byte FIDO application identifier (AID) (A0 00 00 06 47 2F 00 01). After processing this message, the device responds with an R-APDU.

The R-APDU contains the response data followed by the status word. The data field corresponds to the ASCII string “FIDO_2_0”, while the status word 0x9000 indicates successful execution of the command.

3.7.1.4 Chaining Mechanisms

Within ISO-DEP, APDUs are transported using I-Blocks. Since the maximum size of a single ISO-DEP frame is limited by the negotiated frame size, APDUs exceeding this size must be fragmented. This is achieved through **ISO-DEP chaining**, where a single APDU is split across multiple I-Blocks and reassembled by the receiving side.

In addition to transport-level fragmentation, large responses may also be handled at the application level. In the case of short APDUs, the response length is limited by the

value of the Le field. If the response exceeds this limit, it can be delivered in multiple steps using the status word mechanism (e.g., SW1 = 0x61), where the client issues the GET RESPONSE command to retrieve the remaining data.

The two chaining mechanisms operate at different layers: ISO-DEP chaining ensures reliable transmission of a single APDU over multiple frames, while application-level response handling enables large response data to be retrieved through multiple APDU commands, especially when short APDU length fields are used.

■ 3.7.2 NFC-DEP

The **NFC-DEP** (NFC Data Exchange Protocol), defined in [18], is designed for peer-to-peer communication between two **active** NFC devices. Unlike ISO-DEP, which follows a reader–card model, NFC-DEP enables symmetric communication, allowing both devices to alternate between initiator and target roles.

In contrast to ISO-DEP, **NFC-DEP does not utilize APDUs** and does not follow a strict command–response model. Instead, communication is based on **message-oriented data exchange**, making it more suitable for peer-to-peer scenarios rather than card emulation.

Since CTAP over NFC relies on APDU communication over ISO-DEP, NFC-DEP is outside the scope of this implementation.

Chapter 4

Implementation

In this chapter, the **technologies used** in the implementation of NFC for the LionKey security key as well as the **software implementation** are described. The reasons for system design decisions come mainly from facts stated in Chapter 3.

4.1 NFC technologies used

After evaluating the available NFC operating modes and specifications, it becomes evident that the Card Emulation Mode is the only mode compatible with the intended use case. The objective is for the device to behave as a passive entity, analogous to a contactless smart card, rather than as an active initiator.

In the ideal case, card emulation based on Secure Element (SE) would be used. However, this is not feasible with respect to the existing LionKey hardware architecture. This approach requires the presence of a dedicated SE capable of isolated execution and secure key storage, which is not available in the current design. Consequently, the implementation must be based on host-based card emulation, where all protocol handling is performed by the microcontroller (MCU).

The required functionality is achieved most appropriately through the emulation of a Type 4 NFC Forum Tag. In contrast to simpler tag types, Type 4 Tag operation is based on the ISO-DEP protocol and supports structured command–response communication via APDUs. This enables the implementation of a state machine and allows for computation within the tag. These properties are essential for authentication mechanisms such as FIDO2. (See 3.1)

NFC-A was selected over NFC-B for the implementation due to its greater support across mobile operating systems.

Our implementation follows the FIDO CTAP2.2 standard [19].

4.2 Hardware

The hardware chosen for the implementation is the **STEVAL-D25R3916B**, a daughter board of the STEVAL-25R3916B platform manufactured by STMicroelectronics [20]. This kit is an evaluation board for the 25R3916B NFC chip, which turned out to be the best option for the purpose of this thesis.

The 25R3916B chip belongs to the STMicroelectronics universal NFC device family. It supports all main NFC operating modes, including reader/writer mode, peer-to-peer communication, and card emulation; however, it does not provide support for secure element–based card emulation.

Unlike dedicated NFC tag ICs with fixed memory structures, the chip operates as a low-level RF frontend, exposing frame-level control to the application layer. Thanks to this, it enables software-driven card emulation, allowing the host microcontroller to implement the protocol logic.

4.2.1 Board Specifications

The STEVAL-D25R3916B embeds the **ST25R3916 chip** with an antenna etched on the PCB and the related Very High Bit Rate (VHBR) tuning circuit, allowing it to communicate at higher speeds.

In terms of host communication, the board provides both Serial Peripheral Interface (SPI) and Inter-Integrated Circuit (I2C) interfaces, complemented by an interrupt request line (IRQ) used to signal events to the MCU.

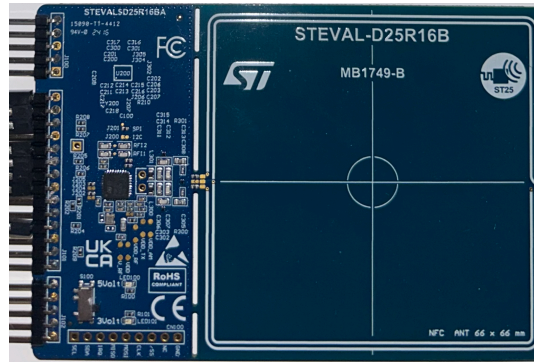


Figure 4.1. The STEVAL-D25R3916B board.

4.2.2 Note: Energy Harvesting

A significant limitation of the selected device is its absence of energy harvesting capabilities. Consequently, the security key cannot operate as a fully passive, standalone authenticator and requires an external power source during operation. It is important to note that achieving true standalone functionality would require not only an NFC frontend capable of efficient energy harvesting but also a substantially lower-power MCU.

The power demands of the current hardware platform exceed what can be reliably supplied by the electromagnetic field generated by typical NFC readers, making passive operation impossible under these conditions.

4.3 Middleware

The middleware used in this project is the STSW-ST25RFAL002 package, provided by STMicroelectronics as an implementation of the RFAL (RF Abstraction Layer) software for the ST25R3916B NFC frontend. RFAL represents a hardware-oriented abstraction layer that encapsulates low-level device control and RF protocol handling. It provides a set of primitives for device initialization, RF configuration, register-level access, and data exchange.

In our implementation, RFAL forms the intermediary layer between the LionKey application and the NFC frontend. The application exchanges data and control requests with RFAL, while RFAL is responsible for direct interaction with the NFC device, low-level NFC control, communication parameters negotiation, subsequent ISO-DEP R-APDU fragmentation (i.e. when the size APDU being sent is larger than the negotiated frame size) and meeting FWT deadlines (by automatically sending WTX blocks). The higher-level communication logic, including the NFC state machine and protocol flow control, including short APDU chaining, is implemented in the application layer.

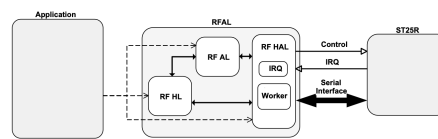


Figure 4.2. Communication flow between the Application (LionKey) and the NFC device using RFAL. (Source: STMicroelectronics STSW-ST25RFAL002 Data Brief)

4.4 Communication with the NFC Frontend

This section describes the communication interface between the MCU and the NFC frontend.

4.4.1 SPI Interface

The communication between the MCU and the NFC frontend is implemented using SPI. Compared to I2C, SPI provides higher data throughput, lower communication latency, and, most importantly, fully deterministic timing due to its synchronous nature and absence of bus arbitration. These properties are critical to ensure protocol compliance. Any excessive delay in data transfer between the front end and the MCU may lead to FWT violations, thus prematurely terminating the NFC session.

In our implementation, SPI operates in a master-slave configuration, with the MCU acting as the master and ST25R3916B acting as the slave device. Chip select (CS) is manually driven via a GPIO pin to ensure precise control over transaction boundaries.

4.4.2 Interrupt Handling

The IRQ line is asserted by the NFC frontend to indicate events such as reception of RF frames, completion of transmission, state changes in the RF field or error conditions.

Upon receiving the interrupt, the MCU reads the corresponding status registers through SPI and sends the event to RFAL. This mechanism enables prompt processing of incoming ISO-DEP frames and immediate reaction to transmission errors.

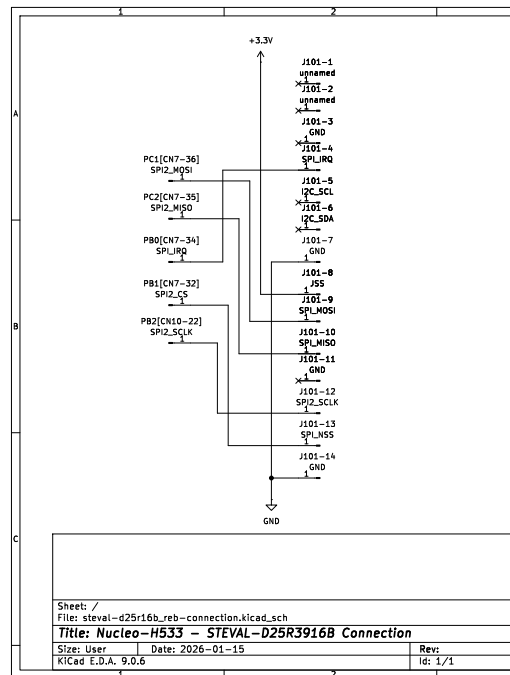


Figure 4.3. KiCad scheme showing how the STEVAL-D25R3916B board is connected to the Nucleo-H533 MCU.

4.5 Software Implementation

In terms of code, the software implementation follows the practices set by Ing. Endler in the original project. It is written in C with no dynamic memory allocations utilizing a layered design that separates the NFC transport layer, APDU processing, and higher-level FIDO logic.

The NFC functionality is implemented using a set of state machines integrated into the existing application architecture, with the NFC worker function being run periodically from the main application loop.

The primary state machine models the operational state of the NFC frontend and distinguishes between the following states:

- Inactive (Idle) state

The device is not within the range of an NFC reader.
- Active state

CE mode is active and data exchange with the reader is in progress.
- Reset state

The communication session has ended and the NFC runtime context is reinitialized to its default configuration.

4.5.1 Transaction Handling

To ensure non-blocking operation and enable concurrent execution of other critical components in parallel with NFC, the card emulation task is implemented as a separate

state machine. This secondary state machine governs the progression of individual NFC transactions, including reception and transmission phases.

Upon receiving an APDU from the reader, the message is parsed and processed according to its semantics. The parser supports all defined APDU formats and handles edge cases according to the mentioned ISO specification. The implementation supports both CTAP2-related communication and basic NDEF communication. The NDEF state machine provides the user with an URI record that directs the user to the lionkey.dev website.

Upon receiving, parsing and, in case of a chained APDU request, concatenating a CTAP2 command, this is then handled by the CTAP API provided in the original LionKey solution.

Management of the communication state machine is performed by periodic polling of the exchange status provided with the RFAL library with errors handled in accordance with the ISO specification.

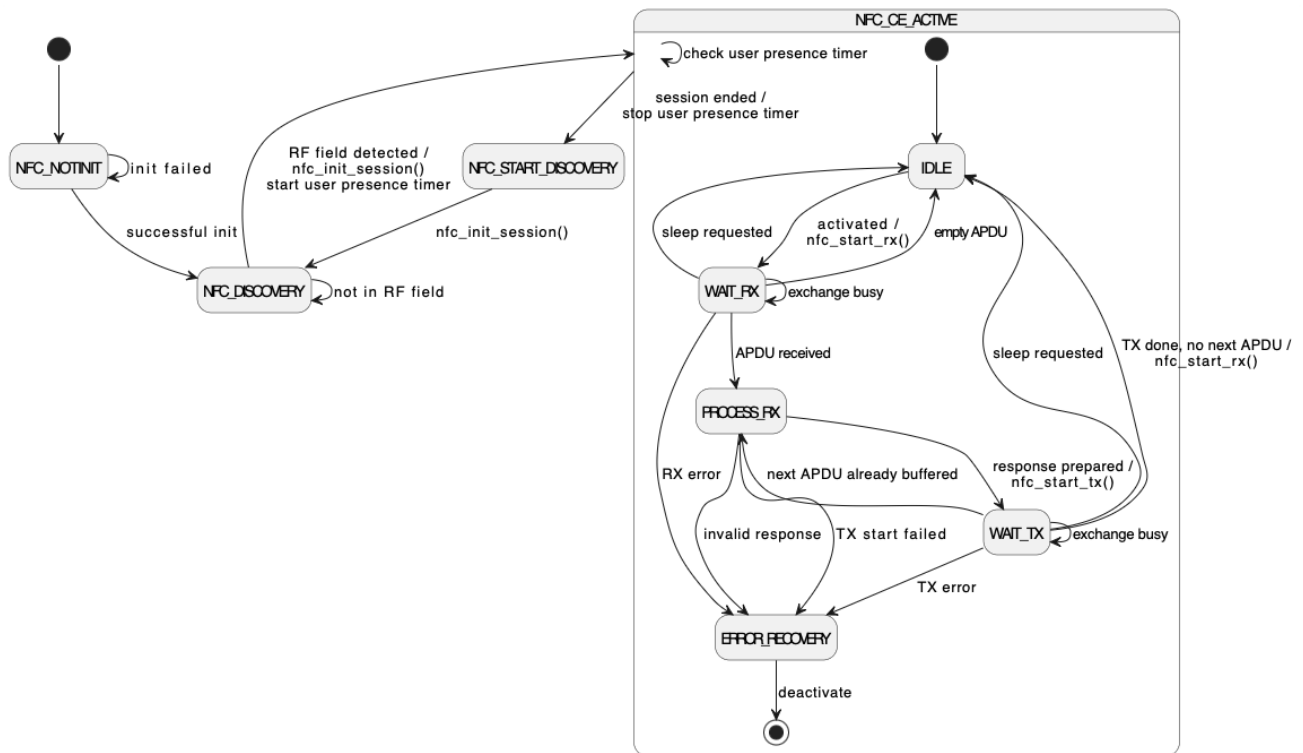


Figure 4.4. PlantUML diagram of the implemented state machine.

Chapter 5

Results and Evaluation

This chapter describes how the correctness of the implementation was tested.

5.1 Methodology

The evaluation process was structured in two stages, corresponding to different layers of the NFC communication stack.

First, the ability of the device to correctly respond during the activation phase was verified. This included transmission of fundamental identification and protocol parameters, namely ATQA (SENS_RES), UID, SAK, and ATS. The tests were performed using an external NFC reader under controlled conditions, enabling detailed observation of the anti-collision and activation sequence. Successful completion of this stage confirms correct operation at the physical and protocol initialization layers as defined by ISO/IEC 14443 and the ISO-DEP protocol.

In the second stage, the communication capabilities of the device were evaluated. Both supported application layers, CTAP and NDEF URI handling, were tested. Interoperability testing was conducted using consumer mobile devices as well as personal computers.

The mobile devices used for testing were multiple iPhone devices ranging from iPhone 11 to iPhone 15 with both iOS 18 and iOS 26, and a Samsung Galaxy S22+, representing the dominant mobile platforms, iOS and Android. Regarding personal computers, Apple does not allow the use of NFC security keys on Mac¹, therefore, the evaluation was conducted using a Windows device.

This allowed for evaluation under realistic usage conditions and comparison with results obtained from the external reader.

The evaluation considered not only the successful command processing, but also the correctness of the protocol, the latency of the response, and the overall success rate of communication exchanges. In addition, selected negative test cases involving unsupported or malformed APDU commands were performed to assess the robustness of the implementation. Particular attention was paid to timing constraints imposed by the ISO-DEP protocol, as these are critical for interoperability with mobile devices.

¹ <https://support.apple.com/en-us/102637>

5.2 Response Testing

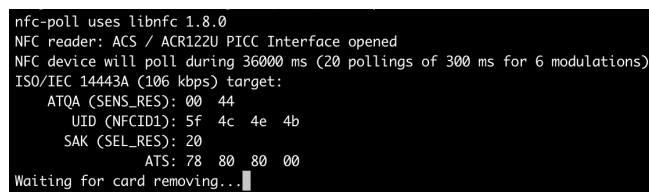
The basic communication parameters of the emulated card were verified using the `nfc-poll` utility from the `libnfc` library. The tool was used with an external NFC reader (ACS ACR122U) to observe the activation sequence.

The following values were obtained during polling:

```
ATQA (SENS_RES): 00 44
UID (NFCID1):    5F 4C 4E 4B
SAK (SEL_RES):   20
ATS:             78 80 80 00
```

- The **ATQA** value (0x0044) indicates a Type A target using a single-size UID and participation in the anti-collision and selection procedure. The reported UID consists of four bytes: 5F 4C 4E 4B, which correspond to the ASCII sequence “_LNK”. This static identifier was intentionally selected for testing and debugging purposes. In a production deployment, a unique identifier should be assigned to each hardware unit.
- The **SAK** (0x20) signals that the device implements the ISO-DEP protocol.
- The **ATS** sets the parameters of the upcoming transaction. The value of the lower nibble of the first byte, 0x78, represents FSCI (FSC Integer–8—means the FSC is 256 bytes). 0x80, as the value of the second byte, represents the bit rate; in this case, it is `ISO-DEP-ATS-TA-SAME-D`, meaning that the same bit rate is expected in both directions. The upper nibble of the third byte specifies 8 as WFI and the lower nibble 0 as SFGI.

Successful retrieval of these parameters demonstrates that the device correctly participates in the NFC-A activation sequence and exposes the protocol capabilities required for the operation of the Type 4 tag.



```
nfc-poll uses libnfc 1.8.0
NFC reader: ACS / ACR122U PICC Interface opened
NFC device will poll during 36000 ms (20 pollings of 300 ms for 6 modulations)
ISO/IEC 14443A (106 kbps) target:
  ATQA (SENS_RES): 00 44
  UID (NFCID1): 5f 4c 4e 4b
  SAK (SEL_RES): 20
  ATS: 78 80 80 00
Waiting for card removing...
```

Figure 5.1. Screenshot of the result of the `nfc-poll` command

5.3 Communication Capabilities Testing

5.3.1 CTAP Handling

The testing of CTAP functionality demonstrated stable and consistent behavior across multiple client platforms, including Apple iOS devices and Windows-based systems. The complete authentication workflow, comprising both credential registration (Make-Credential) and subsequent authentication (GetAssertion), was successfully executed in all evaluated scenarios. The correctness of the implementation was verified using platform-native WebAuthn interfaces and publicly available testing tools, ensuring that the exchanged messages and responses adhered to the specifications defined by the FIDO Alliance.

```

CE: start TX successful
Received APDU (10 bytes):80100000000001040000
NFC: APDU parsed successfully
Handling extended APDU request: CLA=0x80 INS=0x10 P1=0x00 P2=0x00 Lc=1 Le=0
NFC: CTAP cmd=0x04 cbor_len=0
ctap_request: cmd=0x04 params_size=0
CTAP_CMD_GET_INFO
ctap_request: success in 6 ms, response length 176 bytes

```

Figure 5.2. Device log of processing an APDU containing a CTAP_GETINFO command.

For iOS and Windows devices, no deviations were observed from the expected behavior of the CTAP2 protocol, and the implementation exhibited complete interoperability with standard client platforms.

The tests conducted on the evaluated Samsung Galaxy S22+ did not demonstrate successful CTAP2 with respect to NFC interoperability. This may reflect the limitations of the platform, the vendor, or the software-stack rather than the universal Android ecosystem. Experimental evaluation indicates that the interaction is limited to the legacy U2F protocol (Universal Second Factor). While CTAP2 functionality was successfully verified when the LionKey device was connected via USB, attempts to establish CTAP2-based communication over NFC consistently failed. Instead, the Android system, upon receiving a response associated with the U2F protocol, issued commands consistent with this protocol, suggesting a fallback to legacy behavior in accordance with earlier FIDO Alliance specifications. Since LionKey only supports the CTAP2 standard, it is impossible to comply with Android devices at this point.

These observations suggest that, as of April 2026, Android devices do not provide practical support for CTAP2 over NFC for external authenticators, effectively limiting NFC-based communication to U2F compatibility. Although Google introduced the full FIDO2 API for Android in 2025², this support primarily targets platform authenticators, and external authenticators connected via USB and NFC support remains vendor-specific. Since the LionKey implementation targets FIDO2 exclusively and does not implement backward compatibility with U2F, this platform constraint prevents successful interoperability with Android devices in the NFC use case.

5.3.1.1 Latency and Reliability

The response time was measured for the most computationally demanding CTAP command, CTAP_CMD_MAKE_CREDENTIAL. Across 50 consecutive trials, the average response time was 42 ms in the production mode and 270 ms in the debug mode, which is comfortably within the maximum reply time suggested by the FIDO Alliance of 800 ms. The significant increase in debug mode can be attributed primarily to logging overhead and reduced optimization levels, which introduce additional processing delays.

Regarding reliability, it was observed that the initiating device does not consistently detect premature termination of an NFC session. Specifically, when the authenticator is removed from the field of the initiator during an ongoing authentication procedure, the device correctly transitions to an idle state. However, the client application continues to wait for a response and does not recover from the interrupted communication.

As a consequence, subsequent authentication attempts cannot always be initiated, even

² <https://developers.google.com/identity/fido/android/native-apps>

though the user interface continues to indicate that NFC authentication is available. In this state, the NFC reader fails to re-establish communication with the authenticator, and recovery requires a full reset of the client context, typically by reloading the web page.

It was found that the frequency of this issue depends on the browser used. When tested with Safari, the failure occurred consistently after premature session termination. In contrast, Chrome and Firefox exhibited more robust behavior, requiring a page reload in approximately 40% of cases.

These observations suggest limited robustness in session termination and error handling within the client-side NFC stack. However, it cannot be conclusively determined to what extent this behavior is caused by the browser, the underlying operating system, or the implementation of the authenticator.

■ 5.3.2 NDEF Handling

The NDEF functionality was evaluated using a URI record to verify correct Type 4 Tag operation and interoperability with common NFC-enabled devices. The implementation was tested across multiple platforms, including iOS and Android devices, as well as an external NFC reader.

In all the scenarios tested, the NDEF message was successfully detected and processed, resulting in a 100% success rate in 50 of the tests conducted. The communication followed the standard Type 4 Tag access sequence defined by the NFC Forum, consisting of application selection, capability container access, and NDEF file retrieval.

More specifically, the observed APDU flow consisted of the following steps:

1. SELECT command for the NDEF application identifier
2. SELECT command for the Capability Container (CC) file
3. READ BINARY command to retrieve the CC file
4. SELECT command for the NDEF file
5. READ BINARY command to retrieve the NDEF message.

This sequence was executed consistently on all devices tested without deviation.

Response latency was measured at the APDU level in debug mode in 50 trials. The average response time was 25 ms for SELECT commands and 34 ms for READ BINARY commands. The increased latency for READ operations reflects the larger data payload and processing overhead associated with file retrieval. From the user perspective, no perceptible delay was observed during the interaction.

We also verified correct state machine behavior by manually sending APDUs containing both valid and invalid NDEF commands. This included checking proper handling of file selection and data access as well as handling of invalid APDUs.

■ 5.3.3 Use of CTAP on Real WebAuthn-enabled Websites

The implementation was further evaluated in real-world scenarios by testing it on WebAuthn-enabled websites. The authentication process was verified by authenticating on the WebAuthn demo and on Google.

```

CE: start waiting for command
CE: negotiated max frame size: 252
CE: start RX successful
Received APDU (13 bytes): 00A4040007D276000085010100
NFC: APDU parsed successfully
NDEF: NDEF AID selected
Sent APDU (2 bytes): 9000
CE: start TX successful
Received APDU (7 bytes): 00A4000C02E103
NFC: APDU parsed successfully
NDEF: SELECT
Sent APDU (2 bytes): 9000
CE: start TX successful
Received APDU (5 bytes): 00B000000F
NFC: APDU parsed successfully
NDEF: READ
selected CC file
Response: offset=0 to_read=15 src=0 src_len=15 rsp_len=512
Sent APDU (17 bytes): 000F201000100004060001080000009000
CE: start TX successful
Received APDU (7 bytes): 00A4000C020001
NFC: APDU parsed successfully
NDEF: SELECT
Sent APDU (2 bytes): 9000
CE: start TX successful
Received APDU (5 bytes): 00B0000002
NFC: APDU parsed successfully
NDEF: READ
selected NDEF file
Response: offset=0 to_read=2 src=0 src_len=18 rsp_len=512
Sent APDU (4 bytes): 00109000
CE: start TX successful
Received APDU (5 bytes): 00B0000002
NFC: APDU parsed successfully
NDEF: READ
selected NDEF file
Response: offset=2 to_read=16 src=0 src_len=18 rsp_len=512
Sent APDU (4 bytes): 00109000
CE: start TX successful
Received APDU (5 bytes): 00B0000210
NFC: APDU parsed successfully
NDEF: READ
selected NDEF file
Response: offset=2 to_read=16 src=0 src_len=18 rsp_len=512
Sent APDU (18 bytes): D1010C55016C696F6E6865792E6465769000

```

Figure 5.3. Device log of NDEF APDU communication.

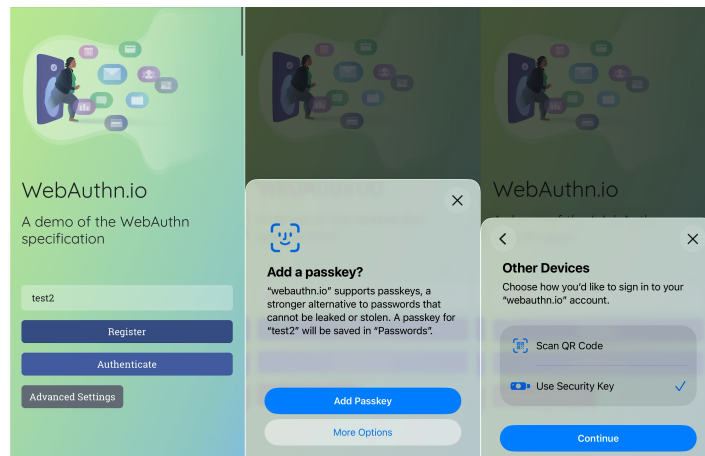


Figure 5.4. WebAuthn LionKey passkey registration flow via NFC on the iPhone.

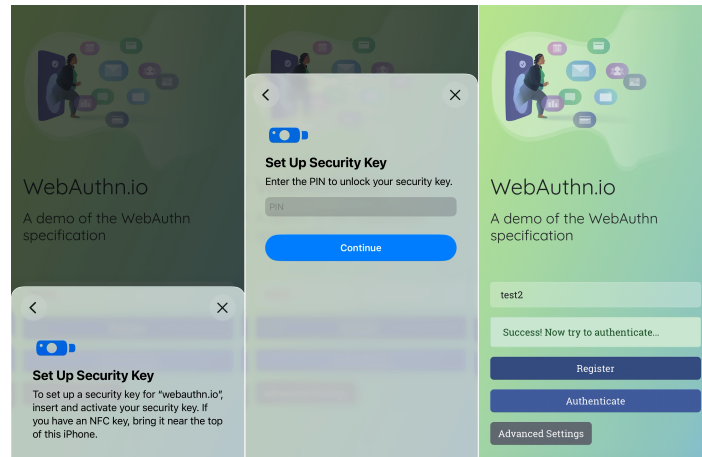


Figure 5.5. WebAuthn LionKey passkey registration flow via NFC on the iPhone.

Chapter 6

Conclusion

This thesis aimed to extend the LionKey hardware authenticator with Near Field Communication (NFC) capabilities, enabling its use in contactless authentication scenarios based on the FIDO2 CTAP standard. To achieve this, an implementation of NFC card emulation using ISO-DEP was designed and integrated into the existing system architecture.

The proposed solution was validated experimentally. As demonstrated in Chapter 5, the implementation supports both CTAP-based communication over NFC and NDEF-based data exchange. Correct functionality was confirmed at the protocol level by testing the APDU processing capabilities and at the application level through interoperability with multiple client platforms, including mobile devices and external NFC readers.

These results come with certain practical constraints worth noting. In particular, the reliability of NFC-based authentication is affected by inconsistent session termination handling on the client side, which may require manual recovery. Furthermore, it was discovered that not all Android devices support the CTAP2 protocol. In addition, the current hardware platform does not support energy harvesting in combination with the required communication capabilities, limiting the feasibility of a fully passive device and ruling out commercial viability in its current form.

Addressing these constraints fully would require a secure element-based platform that would integrate both NFC and cryptographic functionality. Such a transition would necessitate a complete redesign of the current system architecture and is therefore beyond the scope of this thesis.

In conclusion, the objectives of this thesis were met successfully. The presented implementation demonstrates that the integration of NFC-based communication into an existing hardware authenticator is feasible and provides a functional foundation for the further development of contactless security keys.

References

- [1] ITU. *Global and regional ICT data*. March, 2026.
https://www.itu.int/en/ITU-D/Statistics/Documents/facts/ITU_regional_global_Key_ICT_indicator_aggregates_Nov_2025.xlsx.
- [2] Verizon. *Data Breach Investigations Report*. 2025.
<https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf>.
- [3] Jumpcloud. *SME IT Trends Report*. 2024.
<https://jumpcloud.com/wp-content/uploads/2024/02/202401-EB-SMEITTrendsReport.pdf>.
- [4] Martin ENDLER. *FIDO2 USB bezpečnostní klíč*. 2025.
- [5] Sapio Research. *Consumer Password & Passkey Trends*. 2024.
<https://fidoalliance.org/wp-content/uploads/2024/05/World-Password-Day-2024-Report-FIDO-Alliance.pdf>.
- [6] 6sense . *Market Share of Yubico*. 2025.
<https://6sense.com/tech/two-factor-authentication/yubico-market-share>.
Archived at:
<https://perma.cc/MZ78-FJ62>.
- [7] Wikipedia contributors . *Post-quantum cryptography — Wikipedia, The Free Encyclopedia*. 2026.
https://en.wikipedia.org/w/index.php?title=Post-quantum_cryptography&oldid=1349019275. [Online; accessed 22-April-2026].
- [8] Nina Bindel, Cas Cremers, and Mang Zhao. *FIDO2, CTAP 2.1, and WebAuthn 2: Provable Security and Post-Quantum Instantiation*. Cryptology ePrint Archive, Paper 2022/1029. 2022.
<https://eprint.iacr.org/2022/1029>.
- [9] Quentin M. Kniep. *Post-Quantum FIDO2 Security Keys using Hash-Based Signatures*. 2021.
https://sar.informatik.hu-berlin.de/research/publications/SAR-PR-2021-05/SAR-PR-2021-05_.pdf.
- [10] Mouser Electronics. *RFID & NFC, An Introduction to Near Field Communications*. 2026.
https://eu.mouser.com/applications/rfid-nfc-introduction/?srsltid=AfmB0orUoW7dRVaiDpD7tnmaVleDhgMmboEuJqqAB8Rxx17e4JZ_P1CT.
- [11] ISO/IEC. *ISO/IEC 14443-2:2018 Cards and security devices for personal identification — Contactless proximity objects — Part 2: Radio frequency power and signal interface*. 2020.
- [12] ISO/IEC. *ISO/IEC 14443-1:2018 Cards and security devices for personal identification — Contactless proximity objects — Part 1: Physical characteristics*. 2018.

- [13] Erik Hubers. *NFC Protocol Stack*. January 15, 2015.
https://commons.wikimedia.org/wiki/File:NFC_Protocol_Stack.png.
- [14] Yubico. *YubiKey Technical Manual*. 2026.
<https://docs.yubico.com/hardware/yubikey/yk-tech-manual/webdocs.pdf>.
- [15] Google LLC. *Titan Security Key*. 2026.
<https://cloud.google.com/security/products/titan-security-key>.
- [16] ISO/IEC. *ISO/IEC 14443-3:2018 Cards and security devices for personal identification — Contactless proximity objects — Part 3: Initialization and anticollision*. 2018.
- [17] ISO/IEC. *ISO/IEC 14443-4:2018 Cards and security devices for personal identification — Contactless proximity objects — Part 4: Transmission protocol*. 2018.
- [18] ISO/IEC. *ISO/IEC 18092:2023 Telecommunications and information exchange between systems — Near Field Communication Interface and Protocol 1 (NFCIP-1)*. 2023.
- [19] FIDO Alliance. *Client to Authenticator Protocol (CTAP) Proposed Standard*. February 26, 2026.
<https://fidoalliance.org/specs/fido-v2.3-ps-20260226/fido-client-to-authenticator-protocol-v2.3-ps-20260226.html>.
- [20] STMicroelectronics. *STEVAL-25R3916B Data brief*. 2022.
https://www.st.com/resource/en/data_brief/steval-25r3916b.pdf.

Appendix A

Glossary

APDU	■ Application Protocol Data Unit
ASK	■ Amplitude Shift Keying
ATQA	■ Answer To reQuest (nfc-A)
ATS	■ Answer To Select
BPSK	■ Binary Phase Shift Keying
CBOR	■ Concise Binary Object Representation
CE	■ Card Emulation
CID	■ Card Identifier
CL	■ Cascade Level
CLA	■ Class Byte
CTAP	■ Client to Authenticator Protocol
CTU FEE	■ Czech Technical University in Prague, Faculty of Electrical Engineering
DEP	■ Data Exchange Protocol
DID	■ Device Identifier
EM	■ Electromagnetic
fc	■ Carrier Frequency
FGT	■ Frame Guard Time
FIDO	■ Fast IDentity Online Alliance
FSC/FSD	■ Frame Size Card/Device
FWT	■ Frame Waiting Time
HAL	■ Hardware Abstraction Layer
HB	■ Historical Bytes
HID	■ USB Human Interface Devices
IDE	■ Integrated Development Environment
IRQ	■ Interrupt Request
I2C	■ Inter-Integrated Circuit
LLCP	■ Logical Link Control Protocol
MCU	■ Microcontroller Unit
MFA	■ Multi-factor Authentication
NFC	■ Near Field Communication
OOP	■ Object-Oriented Programming
OS	■ Operating System
PCB	■ Printed Circuit Board
PCD	■ Proximity Coupling Device
PICC	■ Proximity Integrated Circuit Card
PQ	■ Post-Quantum
PXD	■ Proximity Extended Device
RATS	■ Request Answer To Select
REQA	■ REQuest, Type A
REQB	■ REQuest, Type B
RF	■ Radio Frequency

RFAL	■ RF Abstraction Layer
RFID	■ Radio-Frequency Identification
SE	■ Secure Element
SPI	■ Serial Peripheral Interface
SW	■ Status Word
UI	■ User Interface
UID	■ Unique IDentifier
USB	■ Universal Serial Bus
UV	■ User Verification
U2F	■ Universal 2nd factor
VHBR	■ Very High Bit Rate
WTX	■ Waiting Time Extension
2FA	■ Second-factor Authentication